

2. Requirements + System Constraint

• What are Requirements?

Requirements define what the system should do and the expectations of users and stakeholders.

They are the foundation of good system design.

• Types of Requirements

- Functional Requirements: What the system should do. (Features, behaviors, API endpoints, etc.)
- Non-Functional Requirements: How well the system should do it. (Performance, scalability, reliability, etc.)

• Example: Ride Sharing App

Functional Requirements
• User registration & login • Book a ride • Match rider with driver • Real-time location tracking • In-app payments • Ride history • Notifications

Non-Functional Requirements
• Support 1M+ users • 99.99% availability • Low latency (< 200ms for API) • Scalable to handle high traffic • Secure user data & payments • Fault tolerant & highly reliable

• Key System Constraints

Constraint	Description
Latency	Time taken to respond to a request. Should be low for good UX.
Throughput	Number of requests the system can handle per second.
Scalability	Ability to handle growth (more users, data, traffic).
Availability	System uptime and readiness to serve requests.
Reliability	System should work correctly even in failures.
Consistency	Data should be accurate and consistent across the system.
Durability	Data should not be lost even in case of failures.
Security	Protect data and ensure only authorized access.
Cost	Optimize infrastructure and operational costs.

• Trade-offs (No Free Lunch)

- Consistency vs Availability (CAP Theorem)
- Performance vs Cost
- Scalability vs Complexity

Note:

Understanding requirements and constraints is the first and most important step in system design.

3. Scalability

• What is Scalability?

Scalability is the ability of a system to handle increasing load by adding resources without degrading performance.

A scalable system can grow with demand.

• Why is Scalability Important?

- Handles more users and data
- Improves performance and user experience
- Ensures business growth
- Cost effective in the long run

• Types of Scalability

1. Vertical Scaling (Scale Up)

- Add more power (CPU, RAM, SSD) to a single server.
- Simple to implement
- Limited by hardware capacity
- Cost becomes high at large scale

2. Horizontal Scaling (Scale Out)

- Add more servers/instances to distribute the load.
- More complex to implement
- Highly available and fault tolerant
- Cost effective at large scale

• Scale Up vs Scale Out

	Scale Up	Scale Out
Architecture		
Cost	High	Low (at scale)
Performance	Limited	High
Fault Tolerance	Low	High
Complexity	Low	High

• Approaches to Achieve Scalability

Load Balancing



Distribute incoming traffic across multiple servers.

Stateless Services



Keep services stateless so any server can handle any request.

Caching



Cache frequently used data to reduce load.

Database Scaling



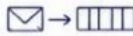
Use replication, sharding, and read replicas.

CDN



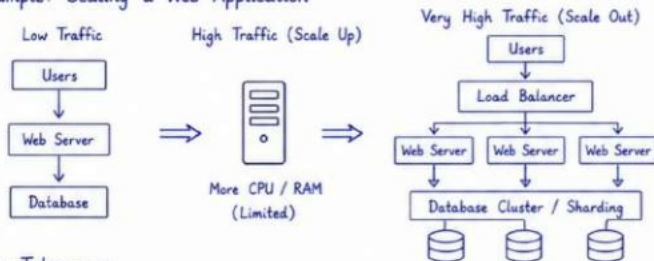
Deliver content from servers closer to the users.

Asynchronous Processing



Use queues and background jobs to handle heavy tasks.

• Example: Scaling a Web Application



Note :

Design for scale from the beginning.
It is easier to scale out than scale up at massive scale.

• Key Takeaways

- Scalability ensures the system can handle growth.
- Horizontal scaling is preferred for modern distributed systems.
- Use the right combination of techniques based on your use case.

4. Availability + Reliability

• What is Availability?

Availability is the measure of how often a system is operational and accessible to users.

It is usually expressed as a percentage of uptime.

$$\text{Availability (\%)} = \frac{\text{Uptime}}{\text{Uptime} + \text{Downtime}} \times 100$$

• Key Difference

	Availability	Reliability
Focus	Minimize downtime	Minimize failures
Goal	Keep system up	Keep system correct
Metric	Uptime %	MTBF, Error rate
Question	"Is the system up when I need it?"	"Does the system work correctly all the time?"

• What is Reliability?

Reliability is the ability of a system to function correctly and consistently over time without failures.

A reliable system performs as expected, even in adverse conditions.

• High Availability vs Fault Tolerance

High Availability (Reduce Downtime)	Fault Tolerance (Handle Failures)
Focus on minimizing downtime	Focus on system's ability to continue working even when parts fail
Achieved through redundancy, failover, replication	Achieved through replication, timeouts, retries, graceful degradation
Example: Active-Passive setup	Example: Replicated services, retries

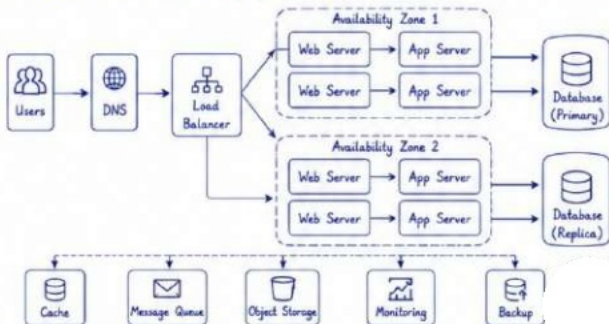
• Ways to Achieve High Availability & Reliability

Redundancy  Remove single points of failure by duplicating components.	Replication  Keep multiple copies of data to prevent data loss.	Load Balancing  Distribute traffic across multiple instances.	Health Checks  Continuously monitor services and restart unhealthy ones.	Automatic Failover  Automatically switch to a healthy instance on failure.	Backups  Regular backups to recover from disasters.	Monitoring & Alerts  Detect issues early and alert the right people.
---	---	---	--	--	---	--

• Common Metrics

- Uptime % = Measure of availability
- MTBF (Mean Time Between Failures) = Average time between failures
- MTTR (Mean Time To Recovery) = Average time to restore service
- Error Rate = Failed requests / Total requests
- SLO / SLA = Service level commitments

• Example Architecture: Highly Available Web Application



Note:

Design for failure, not for success.

Assume everything will fail and

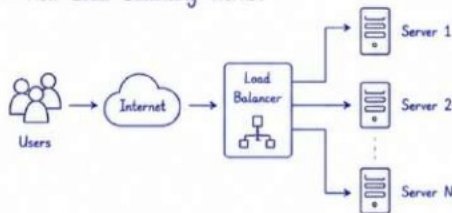
Key Takeaways

5. Load Balancing

• What is Load Balancing?

Load Balancing is the process of distributing incoming network traffic across multiple servers to ensure no single server is overwhelmed. It improves performance, availability and scalability.

• How Load Balancing Works?



• Types of Load Balancers

1. Hardware Load Balancer

- Physical devices.
- High performance and reliability.
- Expensive.
- Example: F5, Citrix ADC.

2. Software Load Balancer

- Runs on standard servers.
- Cost effective and flexible.
- Easy to deploy and scale.
- Examples: Nginx, HAProxy, Envoy.

• Key Features

- ✓ Distributes load evenly across servers
- ✓ Health checks and removes unhealthy servers
- ✓ Exports SSL termination
- ✓ Provides fault tolerance
- ✓ Supports session persistence (sticky sessions)
- ✓ Scalable and highly available

• Example Architecture



Note :

Good load balancing =
Better performance +
High availability +
Happy users! 😊

Key Takeaways

- Load balancing is essential for scalable and reliable systems.
- Use the right algorithm based on your use case.
- Combine with health checks, auto scaling and monitoring for best results.

• Why is Load Balancing Important?

- Prevents any single server from being a bottleneck
- Improves response time and user experience
- Increases system availability and reliability
- Enables horizontal scaling
- Helps in efficient resource utilization

• Where Can Load Balancing Occur?

Client-Side	Logic to distribute requests is in the client (rare).
DNS Load Balancing	Different IPs returned by DNS based on some policy.
Network Load Balancer (L4)	Operates at Transport layer (TCP/UDP), based on IP and Port.
Application Load Balancer (L7)	Operates at Application layer (HTTP/HTTPS), understands content (URL, headers, etc.).

• Common Load Balancing Algorithms

Algorithm	Description
Round Robin (RR)	Requests are distributed sequentially to each server.
Least Connections (LC)	Requests are sent to the server with the least active connections.
IP Hash	Client IP is hashed to a server. Same IP → same server (session stickiness).
Weighted Round Robin (WRR)	Servers have different weights based on capacity.
Least Response Time	Requests are sent to the server with the fastest response time.
Random	Requests are sent to a random server.

6. Caching

• What is Caching?

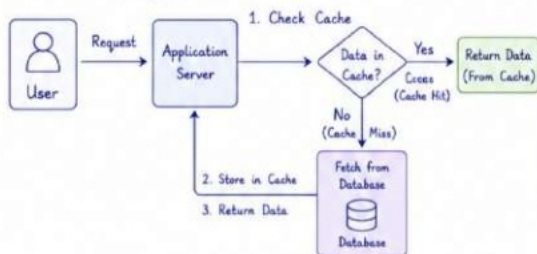
Caching is the process of storing frequently used data in a fast storage (cache) so that future requests for the same data can be served quickly.

It reduces load on slower storage (like databases) and improves system performance.

• Why is Caching Important?

- Reduces latency and improves response time
- Reduces load on database / backend services
- Improves throughput and scalability
- Cost effective (fewer DB queries, less compute)
- Improves overall user experience

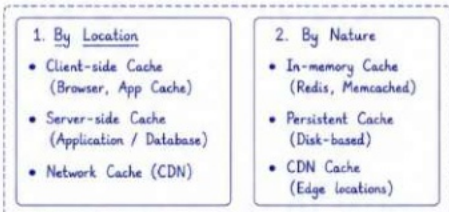
• How Caching Works?



• Where to Use Cache?

Layer	What to Cache (Examples)
Browser / Client	Static assets (JS, CSS, Images)
CDN	Images, videos, static files, API responses
Application Cache	Session data, user preferences, frequently read data
Database Cache	Query results, reference data, expensive computations
Distributed Cache	Shared cache for multiple application instances

• Types of Cache



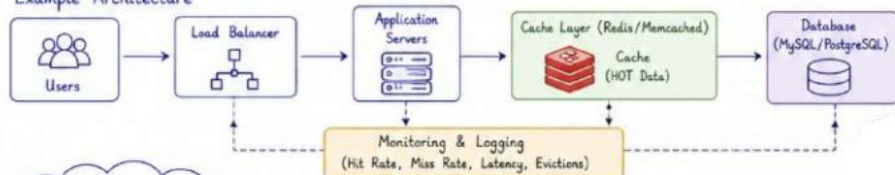
• Common Caching Strategies

Strategy	Description
Cache-Aside (Lazy Loading)	App first checks cache. On miss, fetches from DB, stores in cache, then returns.
Read-Through	Cache layer sits in front of DB. On miss, cache loads from DB automatically.
Write-Through	Write goes to cache and DB at the same time. Cache always has latest data.
Write-Back (Write-Behind)	Write goes to cache first. DB is updated later (async). Faster but risk of data loss.
Refresh-Ahead	Frequently accessed data is refreshed in cache before it expires.

• Cache Eviction Policies

Policy	How it Works
LRU (Least Recently Used)	Evict least recently used items first.
LFU (Least Frequently Used)	Evict least frequently used items.
FIFO (First In First Out)	Evict oldest items first.
TTL (Time To Live)	Evict items that have expired.
Random	Evict random items.

• Example Architecture



Note :

Key Takeaways

7. Databases: SQL vs NoSQL

• What is a Database?

A database is an organized collection of data that can be easily accessed, managed and updated.

• Types of Databases

1. SQL (Relational Databases)

Store data in tables (rows and columns).
Have a fixed schema.
Use SQL for querying.

2. NoSQL (Non-Relational Databases)

Store data in non-tabular formats.
Schema is flexible or dynamic.
Designed for scale and high performance.

• SQL vs NoSQL - Key Differences

Feature	SQL (Relational Databases)	NoSQL (Non-Relational Databases)
Data Model	Table (rows & columns)	Key-Value, Document, Column, Graph
Schema	Fixed (predefined schema)	Flexible (dynamic schema)
Query Language	SQL	Different query APIs (varies by database)
Joins	Supports Joins	Limited or No Joins
Transactions	ACID Transactions	BASE (Basically Available, Soft state, Eventually consistent)
Scalability	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
Performance	Good for complex queries, less for big scale writes	High performance for large scale and high throughput
Consistency	Strong Consistency	Eventual Consistency (varies)
Data Structure	Structured	Semi-structured / Unstructured
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Cassandra, Redis, DynamoDB, HBase
Best For	Complex relationships, transactions, reporting	Big data, high availability, real-time apps

• Types of SQL Databases



RDBMS (Relational DBMS)
MySQL, PostgreSQL, Oracle,
SQL Server



NewSQL Databases
CockroachDB, Google Spanner,
TiDB



Data Warehouse (Analytical)
Snowflake, Amazon Redshift,
BigQuery

• Types of NoSQL Databases



Document Store
Stores data in JSON/BSON documents.
Example: MongoDB, CouchDB



Key-Value Store
Stores data as Key → Value pairs.
Example: Redis, DynamoDB, Riak



Column Store
Stores data in columns, optimized for large datasets. Example: Cassandra, HBase



Graph Database
Stores data as nodes and edges.
Example: Neo4j, Amazon Neptune

• Quick Comparison (Analogy)

SQL is like...



Organized spreadsheets
with rules and
relationships.

NoSQL is like...



Flexible storage
that can adapt to
different needs.

Note:

There is no one-size-fits-all.
Choose the database that
best fits your use case
and scale requirements.



• Real World Examples

Application / System	Better Choice	Why?
Banking System	SQL	Needs ACID, consistency, complex transactions
User Authentication	SQL	Relational data, strong consistency
Social Media Feed	NoSQL (Document/Key-Value)	High write/read, flexible schema
Real-time Analytics	NoSQL (Column Store)	High throughput, large volumes of data
Shopping Cart	NoSQL (Key-Value)	Fast read/write, simple data access
Content Management	NoSQL (Document Store)	Flexible content structure

8. Database Scaling + Sharding

Why Do We Need to Scale Databases?

- As data and traffic grow:
- Single server becomes a bottleneck
- Storage capacity is limited
- High CPU / memory usage
- Affects performance and availability

Two Types of Scaling

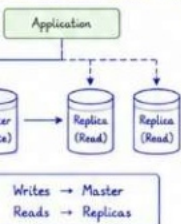
Vertical Scaling (Scale Up)	2. Horizontal Scaling (Scale Out)
- More power (CPU, RAM, SSD) on single server. - Simple to implement - Low downtime (maintenance) - Costly at high scale - Limited by hardware max	- Add more servers and distribute the load. - More complex - High availability - Cost effective at large scale - Near unlimited scale

Vertical Scaling vs Horizontal Scaling

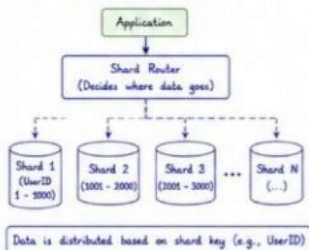
	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
Cost	Increase power of one server	Add more servers
Limit	Hardware limit	Practically unlimited
Expense	Very expensive at high end	Cost effective
Downtime	Required	Minimal / None
Complexity	Low	High
Use Case	Small to medium workloads	Large scale applications

Scaling Examples

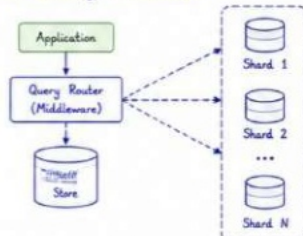
1. Read Replicas (Scale Out)



2. Sharding Example



• Sharding Architecture



- Query Router:
- Finds correct shard based on shard key
 - Sends query to shard
 - Aggregates results (if needed)
- Config Store:
- Keeps mapping of shard key to shard

Note:

- Vertical scaling is not just adding servers.
- Horizontal scaling requires careful design, monitoring, and handling data distribution, consistency and failures.

Key Takeaways

- ✓ Vertical scaling is easier but has a limit.
- ✓ Horizontal scaling is the key to handle massive scale.
- ✓ Sharding helps in distributing data and write load.
- ✓ Use read replicas, caching and partitioning along with sharding for best results.
- ✓ Choose the right strategy based on your workload and growth.

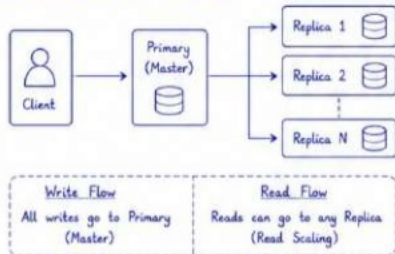
9. Replication + Consistency

Why Replication?

- Improves availability and fault tolerance
- Handles read scale
- Protects against data loss
- Enables data to be closer to users (lower latency)

Replication

Replication means maintaining multiple copies of the same data on different servers.



What is Consistency?

Consistency means all users see the same data at the same time.

In distributed systems, achieving consistency is hard because updates take time to propagate across replicas.

Types of Replication

Type	Description	Use Case
Master-Slave (Primary-Replica)	One primary handles writes. Replicas handle reads.	Most common, simple to implement
Master-Master (Multi-Primary)	Multiple masters accept writes. Data is synced between them.	High availability, geo-distribution
Synchronous Replication	Write is considered successful only after replicas acknowledge.	Strong consistency required
Asynchronous Replication	Write is considered successful after primary writes. Replicas catch up later.	High performance, eventual consistency

Consistency Models

Model	Description	Example Systems
Strong Consistency	All clients see the latest write immediately.	Traditional RDBMS (Single DB)
Eventual Consistency	All replicas will become consistent over time.	Cassandra, DynamoDB, CouchDB
Causal Consistency	If X happened before Y, all users see X before Y.	MongoDB (Causal), Riak
Read-your-writes	User sees their own writes.	Most systems with session guarantees
Monotonic Reads	If a user reads X, they won't see older data.	Dynamo, Spanner
Linearizability	Seems like operations happen in real-time order.	Zookeeper, etcd, Spanner (strong)

Consistency Trade-offs (CAP Theorem)

In distributed systems, you can have only two out of three:

C

Consistency

All nodes see same data

A

Availability

System remains available

P

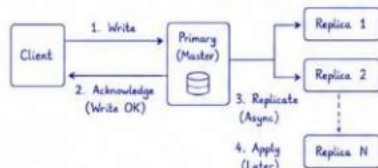
Partition Tolerance

System works despite network partitions

Examples:

- RDBMS (single region): CP (Consistency + Partition Tolerance)
- Cassandra/DynamoDB: AP (Availability + Partition Tolerance)
- MongoDB (default): AP
- Spanner/etcd: CP

How Replication Works (Asynchronous Example)



Note:

Replication improves availability and performance but introduces complexity in consistency.

Key Takeaways

- Replication = multiple copies of data to improve availability and scale reads.
- Primary handles writes; replicas handle reads (in most systems).

Choosing the Right Strategy

Goal	Recommended	Why?
High availability, low latency reads	Asynchronous Replication + Read Replicas	Better performance, eventual consistency is ok
Strong consistency, critical data	Synchronous Replication or Strong Consistency	Ensures latest data everywhere
Global geo-distribution	Multi-Region Replication (Eventual Consistency)	Better user experience, handles partitions
Analytics / Reporting	Read Replicas	Offload reads from primary

10. CAP Theorem + Distributed Systems

What is CAP Theorem?

In a distributed system, you can have at most two out of the following three guarantees:



In real world, network partitions are unavoidable, so systems choose between Consistency and Availability.

Understanding Each Property

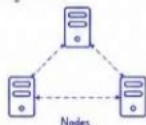
Consistency (C) 	- All nodes return the latest data. - No conflicting updates. - Strong consistency.
Availability (A) 	- Every request receives a response. - System does not refuse requests. - Response may be stale.
Partition Tolerance (P) 	- System continues to operate despite network failures. - Messages may be lost or delayed.

What is a Distributed System?

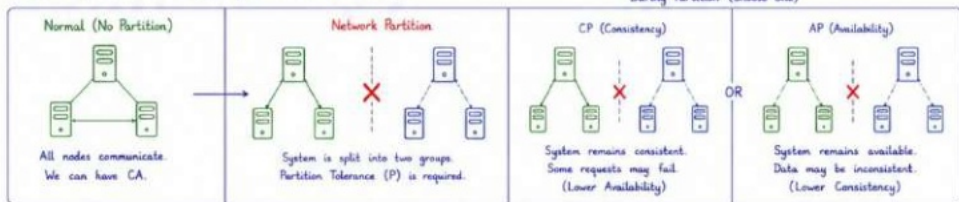
A distributed system is a collection of independent computers (nodes) that appear to users as a single system.

Characteristics

- ✓ No shared memory
- ✓ Nodes communicate over network
- ✓ Concurrency of operations
- ✓ Partial failures are possible
- ✓ Scalability and fault tolerance



CAP Theorem in Action (Example: Network Partition)



Design Considerations

- ✓ Understand your use case
- ✓ What is more important: Consistency or Availability?
- ✓ What types of failures can you tolerate?
- ✓ How to handle retries and timeouts?
- ✓ Monitor, test and plan for failures

Key Takeaways

- CAP Theorem states that in the presence of network partition, you can have only Consistency (C), Availability (A) or Partition Tolerance (P).
- Most distributed systems choose AP because partition tolerance is a must.
- For critical data, CP systems are preferred.
- Understand the trade-offs and choose the right system for your needs.
- Distributed systems are complex; design for failure from day one.

CAP Theorem - Possible Choices

Choice	Description	Example Systems
CP	Consistency + Partition Tolerance System remains consistent even if some nodes are unavailable. Availability is sacrificed.	HBase, MongoDB (w majority), Cassandra (consistency > 1), Zookeeper, Spanner
AP	Availability + Partition Tolerance System remains available even if some data is inconsistent. Consistency is sacrificed.	Cassandra (Consistency ONE), DynamoDB, CouchDB, Riak, Voldemort
CA	Consistency + Availability Only possible when there is no network partition. Partition Tolerance is sacrificed.	RDBMS (MySQL, PostgreSQL) in single region, Redis (single instance)
Can't have CP + A	In a partition, either Consistency or Availability must be sacrificed.	-

Real World Examples

System / Service	CAP Choice	Why?
Google Spanner	CP	Strong consistency across regions is critical. Availability can be traded off.
Cassandra	AP	Designed for high availability and partition tolerance. Eventual consistency.
DynamoDB	AP	High availability and low latency at scale. Offers eventual consistency options.
MongoDB	CP (default)	With majority writes, prefers consistency. Can be tuned for availability.
RDBMS (Single Region)	CA	In single data center, partitions are rare. So Consistency + Availability possible.

Challenges in Distributed Systems

- ✓ Network failures and partitions
- ✓ Node failures
- ✓ Data consistency
- ✓ Time synchronization
- ✓ Concurrency control
- ✓ Scalability
- ✓ Security

Failures are normal, not exceptional!
Design for failure.



11. API Design + REST

• What is an API?

An API (Application Programming Interface) is a set of rules that allows one application to communicate with another.

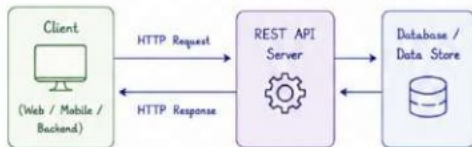
• What is REST?

REST (Representational State Transfer) is an architectural style for building APIs using HTTP methods. It is simple, scalable and stateless.

Key Principles of REST

- ✓ Client-Server
- ✓ Uniform Interface
- ✓ Stateless
- ✓ Layered System
- ✓ Cacheable
- ✓ Code on Demand (optional)

• REST API Architecture



• REST API Example

Resource: /users

Method	Endpoint	Description	Example
GET	/users	Get all users	GET /users
GET	/users/{id}	Get user by ID	GET /users/123
POST	/users	Create a new user	POST /users
PUT	/users/{id}	Update user (replace)	PUT /users/123
PATCH	/users/{id}	Update user (partial)	PATCH /users/123
DELETE	/users/{id}	Delete user	DELETE /users/123

• HTTP Methods

Method	Purpose	Idempotent	Safe
GET	Retrieve a resource	Yes	Yes
POST	Create a new resource	No	No
PUT	Update / Replace a resource	Yes	No
PATCH	Partially update a resource	No	No
DELETE	Delete a resource	Yes	No

• Safe: Does not modify data

• Idempotent: Same request multiple times has same effect

• URL (URI) Design Best Practices

- ✓ Use nouns, not verbs → /users (not /getUsers)
- ✓ Use plural nouns → /users
- ✓ Use hierarchical structure → /users/123/orders/456
- ✓ Use lowercase letters
- ✓ Use hyphens (-) to separate words
- ✓ Avoid file extensions → /users (not /users.json)
- ✓ Use query parameters for filtering, sorting, pagination

Examples:

```
/users?role=admin&status=active&page=2&limit=20  
/orders?userId=123&sort=createdAt&desc
```

• Request & Response Example

Request
GET /users/123
Headers:
Accept: application/json
Authorization: Bearer <token>
Response
Status: 200 OK
Headers:
Content-Type: application/json
Body (JSON):
{
 "id": 123,
 "name": "Nikhil",
 "email": "nikhil@example.com",
 "role": "admin"
}

Error Response Example

Status: 404 Not Found
Headers:
Content-Type: application/json
Body (JSON):
{
 "error": "User not found",
 "message": "No user with id 999",
 "status": 404
}

• Status Codes (Common)

Code Range	Code	Meaning	Use Case
2xx (Success)	200	OK	Request successful
	201	Created	Resource created
	204	No Content	Success, no content to return
3xx (Redirection)	301	Moved Permanently	Resource moved permanently
	304	Not Modified	Used with caching
	400	Bad Request	Invalid request
4xx (Client Error)	401	Unauthorized	Authentication required
	403	Forbidden	No permission
	404	Not Found	Resource not found
	429	Too Many Requests	Rate limit exceeded
5xx (Server Error)	500	Internal Server Error	Something broke
	503	Service Unavailable	Server temporarily down

• API Design Best Practices

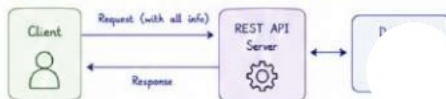
- ✓ Keep it simple and consistent
- ✓ Use meaningful and versioned URLs (/v1/users)
- ✓ Follow HTTP methods correctly
- ✓ Use proper status codes
- ✓ Support pagination, filtering, sorting
- ✓ Implement authentication & authorization
- ✓ Validate input and handle errors gracefully
- ✓ Provide clear documentation
- ✓ Ensure security (HTTPS, rate limiting, CORS)

Versioning Best Practices

- URI Versioning:
/v1/users
- Header Versioning:
Accept: application/vnd.api.v1+json
- Query Parameter Versioning:
/users?version=1

• Statelessness in REST

Each request from the client to the server must contain all the information needed to understand the request. The server does not store any client context between requests.



12. Message Queues + Event-Driven Archit

Why Message Queues?

- ✓ Decouple components and services
- ✓ Handle traffic spikes (buffering)
- ✓ Improve reliability & fault tolerance
- ✓ Enable asynchronous communication
- ✓ Build scalable, flexible systems

What is a Message Queue?

A message queue (or messaging system) is a middleware that allows applications to send, store, and receive messages asynchronously.



How Message Queues Work

1. Producer sends a message to the queue.
2. Queue stores the message until a consumer retrieves it.
3. Consumer reads and processes the message.
4. Message is acknowledged (deleted) after successful processing.



Message Queue Patterns

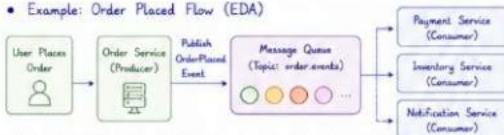
Pattern	Description	Use Case
Point-to-Point (Queue)	One message is consumed by one consumer.	Task distribution, background jobs
Publish/Subscribe (Topic)	One message is delivered to multiple subscribers.	Notifications, real-time updates, fan-out
Competing Consumers (Queue)	Multiple consumers compete to consume messages.	Load balancing across workers
Request/Reply	Producer sends request and waits for reply message.	RPC style communication async

Delivery Guarantees

Guarantee	Description	Example Systems
At-most-once	Message may be lost, but never delivered twice.	Rarely used
At-least-once	Message will be delivered at least once (duplicates possible).	RabbitMQ, Kafka, SQS, Pub/Sub
Exactly-once	Message will be delivered exactly once.	Kafka (idempotent producer + transactions), Pulsar

Note: Use idempotent consumers to handle duplicates safely.

Example: Order Placed Flow (EDA)

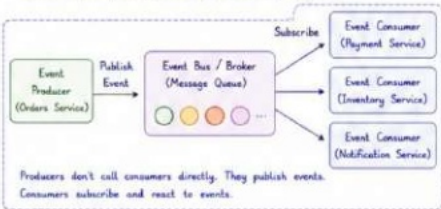


Benefits

- ✓ Services are independent
- ✓ System is resilient to failures
- ✓ Easier to scale each service
- ✓ New consumers can be added without changing producers

Event-Driven Architecture (EDA)

In EDA, services produce events and other services react to those events. Communication happens through events.



Common Use Cases

- Order Processing
- Email / Notification Sending
- Log / Audit Processing
- Video / Image Processing
- Analytics & Monitoring
- Retry & Failure Handling
- Data Pipeline / ETL
- IoT Event Ingestion

Popular Message Queue / Broker Systems

System	Type	Language / Protocol	Highlights
RabbitMQ	Broker	AMQP	Mature, reliable, rich features, routing, ack, DLQ
Kafka	Log-based Broker	TCP	High throughput, durable, streams, replay, partitions
Apache Pulsar	Broker	TCP	Multi-tenancy, geo-replication, separates storage & compute
Amazon SQS	Managed Queue	HTTPS	Fully managed, simple, scales automatically
Google Pub/Sub	Managed Pub/Sub	gRPC / HTTPS	Globally distributed, at-least-once delivery
Redis Streams	Stream	RESP	Lightweight, in-memory, good for real-time

Key Concepts

- Message: The data sent from producer to consumer.
- Acknowledgment (Ack): Confirms message has been processed.
- Visibility Timeout: If not acked in time, message becomes visible again.
- Retry: Failed messages can be retried automatically.
- Dead Letter Queue (DLQ): Messages that fail repeatedly are moved to DLQ.
- Partition: Splits data for parallel processing and scalability (e.g., Kafka).

Best Practices

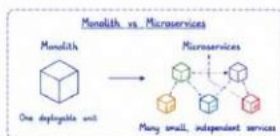
- ✓ Keep messages small and focused.
- ✓ Use meaningful event names and version them.
- ✓ Design consumers to be idempotent.
- ✓ Handle failures with retries and DLQ.
- ✓ Monitor queue length, lag, failures.
- ✓ Secure your message broker (auth, encryption).



13. Microservices + Service Communication

What are Microservices?

- A microservices architecture structures an application as a collection of small, independent services.
- ✓ Each service implements a specific business capability
 - ✓ Services are independent and loosely coupled
 - ✓ Built, deployed and scaled independently
 - ✓ Communicate over a network using lightweight protocols



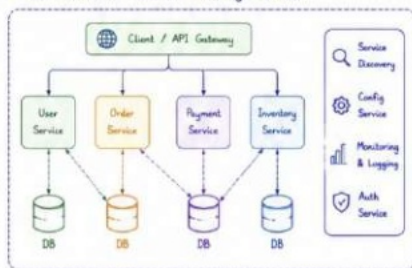
Benefits of Microservices

- ✓ Independent deployment
- ✓ Technology flexibility
- ✓ Scalability (scale what you need)
- ✓ Fault isolation
- ✓ Faster development & delivery

Challenges

- ⚠ Complex distributed system
- ⚠ Network latency & failures
- ⚠ Data consistency
- ⚠ Monitoring & debugging
- ⚠ More DevOps & operational overhead

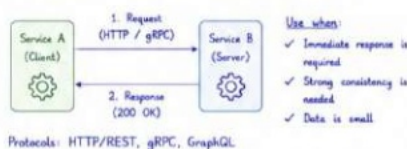
Microservices Architecture (High Level)



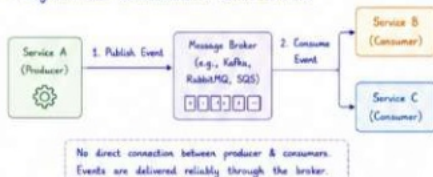
Service Communication Styles

Style	Description	Communication	Use Cases	Pros	Cons
Synchronous	Services wait for immediate response	Request/Response (HTTP/gRPC)	Real-time operations, Get data, Update	Simple, Easy to understand	Coupling, Cascades failures, Latency
Asynchronous	Services communicate via events/messages	Message Broker (Queue/Stream)	Event notifications, Background jobs	Decoupled, Resilient, Scalable	Complexity, Eventual Consistency

Synchronous Communication (Request/Response)



Asynchronous Communication (Event-Driven)



Use when:

- ✓ Multiple services need the data
- ✓ Operations are long-running
- ✓ System should be resilient & decoupled
- ✓ Eventual consistency is acceptable

Common Communication Protocols

Protocol	Type	Description	Best For
REST (HTTP/JSON)	Sync	Widely used, human readable, easy to integrate	General purpose, public APIs
gRPC	Sync	High performance, HTTP/2, Protocol Buffers (binary)	Internal microservices communication
GraphQL	Sync	Client asks for what it needs, reduces over/under fetching	Complex queries, client flexibility
Messaging (Kafka, RabbitMQ, SQS)	Async	Event-based, decoupled communication	Events, notifications, background jobs

Rule of Thumb:

- Use Sync for commands/queries that need immediate result
- Use Async for events, notifications, and long running tasks

Service Discovery

Services don't know each other's location. Service Registry helps!



Types:

- Client-side discovery
- Server-side discovery (via API Gateway)

API Gateway

Single entry point for clients. Handles routing, authentication, rate limiting, etc.



Data Management

- ✓ Each service owns its data (Database per service)
- ✓ Never share database between services
- ✓ Use events or APIs to share data
- ✓ Leads to eventual consistency



Best Practices

Design Around Business Capabilities Create services around business domain, not	Keep Services Small Small services are easier to develop, deploy and maintain	Independent Deployments Deploy services independently using CI/CD pipelines	Resilience Implement timeouts, retries, circuit breakers, bulkheads (e.g., Resilience4j)	Observability Centralized logging, metrics, tracing and monitoring	Security Secure service-to-service communication (mTLS, OAuth2/JWT)	Versioning Version 1, avoid breaking changes
---	---	---	--	--	---	--

14. Storage + CDN

Why Storage?

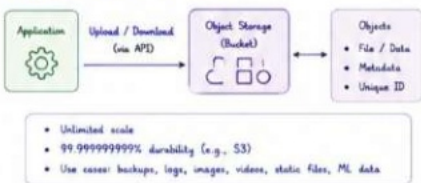
Storage is where your application data, files, and media live. Choosing the right storage ensures durability, availability, performance, and low cost.



Types of Storage

Type	What it Stores	Characteristics	Examples / Use Cases
Block Storage	Raw blocks of data	Low-level, high performance, used by servers/DBs	EBS (AWS), Persistent Disk (GCP)
File Storage	Files in a hierarchical structure	Shared access, easy to organize	EFs (AWS), Azure File, NFS
Object Storage	Objects (data + metadata) in buckets	Highly scalable, durable, access via API	S3 (AWS), GCS, Azure Blob
Database Storage	Structured data in tables/documents	Optimized for queries and transactions	MySQL, PostgreSQL, MongoDB, DynamoDB

Object Storage (How it Works)



Database Storage (Quick View)

Category	Examples	Best For	Notes
Relational (SQL)	MySQL, PostgreSQL, SQL Server	ACID transactions, complex queries	Strong consistency
NoSQL - Document	MongoDB, CouchDB	Flexible schema, rapid development	JSON-like documents
NoSQL - Key Value	Redis, DynamoDB	High throughput, caching, sessions	Key → Value
NoSQL - Wide Column	Cassandra, HBase	Large scale writes, time-series	Horizontal scaling
NoSQL - Graph	Neo4j, Amazon Neptune	Relationships, connected data	Graph queries

Storage Best Practices

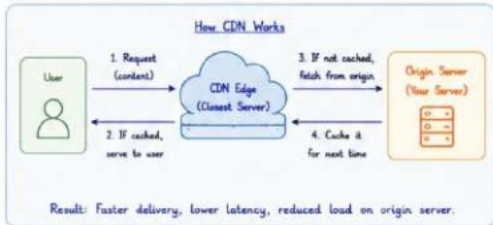
- ✓ Choose the right storage for the right data (don't use SQL for files).
- ✓ Enable encryption at rest and in transit.
- ✓ Use replication / multi-AZ for durability.
- ✓ Lifecycle policies for cost optimization (e.g., move old data to cold storage).
- ✓ Monitor usage, latency, errors and set alerts.

Note

Storage is about durability and cost.
CDN is about speed and availability (closer to users). Both are critical for scalable systems! 🌟

What is a CDN?

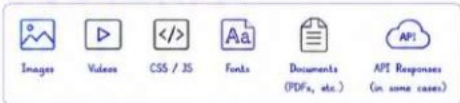
A CDN (Content Delivery Network) is a distributed network of edge servers that cache and deliver content closer to users.



Benefits of CDN

- ✓ Low latency - content served from nearest edge location
- ✓ High availability - CDN absorbs traffic spikes
- ✓ Reduced bandwidth cost - fewer requests hit origin server
- ✓ Better user experience - fast loading of static & dynamic content

What Can Be Delivered via CDN?



Popular CDNs

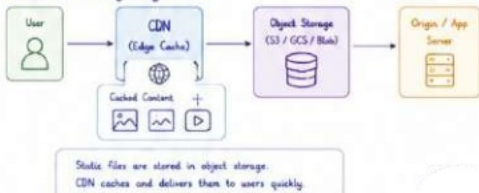
CDN	Provider	Highlights
CloudFront	AWS	Deep AWS integration, secure, global edge locations
Cloud CDN	Google Cloud	High performance, works with GCP services
Azure CDN	Microsoft Azure	Integration with Azure, global reach
Fastly	Fastly	Developer friendly, real-time config, edge compute
Akamai	Akamai	Large network, enterprise grade, very reliable
Cloudflare CDN	Cloudflare	Easy to use, DDoS protection, free tier

When to Use CDN?

- ✓ Your users are global or far from your servers.
- ✓ You serve static or cacheable content (images, videos, files, APIs).
- ✓ You want to offload traffic from your origin servers.
- ✓ You need better performance, reliability and security.



CDN + Storage Together



15. Authentication + Authorization

• What is Authentication?

Authentication verifies who a user is.
It answers: "Are you who you claim to be?"

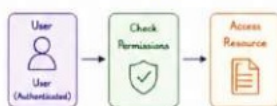


• Common Authentication Methods

Method	Description
Username & Password	Traditional method. Use strong password hashing (bcrypt, Argon2).
Multi-Factor Authentication (MFA)	Add an extra factor (OTP, SMS, Authenticator app, etc.).
OAuth 2.0 / OpenID Connect	Standard protocols for delegated authentication (Google, GitHub, etc.).
API Keys	Simple token-based auth for service-to-service.
Biometrics	Fingerprint, Face ID, etc.

• What is Authorization?

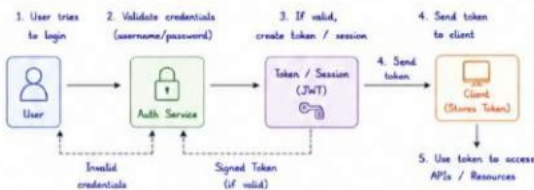
Authorization determines what an authenticated user is allowed to do.
It answers: "What are you allowed to do?"



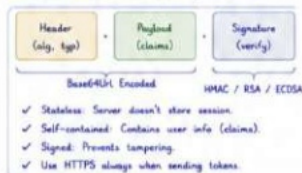
• Key Difference

	Authentication	Authorization
Purpose	Verify identity	Verify permissions
Question	Who are you?	What can you do?
Done by	Auth System	App / Resource Server
Occurs	Once (at login)	Every time (for access)
Example	Login with password	Can user delete this post?

• Authentication Flow (Example)



• JSON Web Token (JWT) Overview



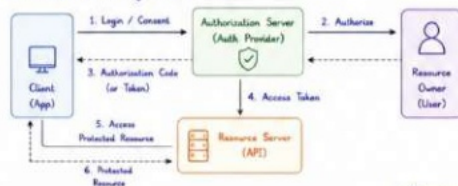
Common Claims

- sub - Subject (user id)
- iss - Issuer
- exp - Expiry time
- iat - Issued at
- aud - Audience

• Authorization Models

Model	Description	Use Case
RBAC (Role-Based Access Control)	Permissions are assigned to roles. Users get roles.	Admin, Editor, Viewer roles
ABAC (Attribute-Based Access Control)	Access based on attributes (user, resource, action, env).	Complex rules, dynamic environments
ACL (Access Control List)	Permissions listed for users or resources.	Simple systems, legacy apps

• OAuth 2.0 (High Level Flow)



• Protecting APIs

- ✓ Use HTTPS everywhere
- ✓ Validate & verify tokens on every request
- ✓ Enforce authorization checks (least privilege)
- ✓ Short-lived access tokens
- ✓ Use refresh tokens securely
- ✓ Rate limiting & brute-force protection

Never trust the client.
Always verify on the server.
Principle of Least Privilege is the key!

• Best Practices

- Use strong password policy (or better: passwordless + MFA).
- Hash passwords with salt (bcrypt / Argon2).
- Use short-lived tokens and refresh tokens.
- Store tokens securely (HttpOnly cookies or secure storage).
- Log authentication events and monitor suspicious activity.
- Don't expose sensitive info in tokens.

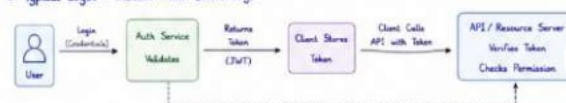
• Common HTTP Status Codes

Code	Meaning	When to Use
401 Unauthorized	Authentication required	No/invalid token, login required
403 Forbidden	Access denied	Authenticated but not allowed
404 Not Found	Resource not found	Hide existence of resource
409 Conflict	Conflict in request	E.g., duplicate data
429 Too Many Requests	Rate limit exceeded	Too many requests
500 Internal Server Error	Server error	Something went wrong

Quick Checklist

- ✓ Is authentication required for this system?
- ✓ Which authentication method will you use?
- ✓ How will you issue and manage tokens/sessions?
- ✓ What authorization model fits your needs?
- ✓ Are APIs properly protected?
- ✓ Are security best practices followed?

• Typical Login + Access Flow (Summary)



16. Rate Limiting + Security

Why Rate Limiting?

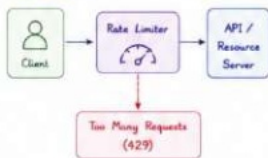
- ✓ Prevent abuse, brute force, and DDoS attacks
- ✓ Ensure fair usage of system resources
- ✓ Protect downstream services
- ✓ Maintain system stability & availability

Rule of Thumb

Rate limiting protects your system.
Security protects your data.
You need both.

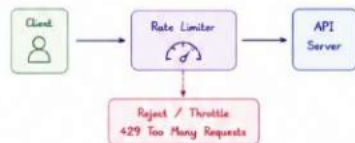
What is Rate Limiting?

Rate limiting restricts the number of requests a client can make in a given time window.



How Rate Limiting Works

1. Client sends request
2. Rate limiter checks the rule
- 3a. If within limit → allow request
- 3b. If over limit → block / reject



Common Rate Limiting Algorithms

Algorithm	How it Works	Pros	Cons	Best For
Fixed Window Counter	Count requests in fixed time window	Simple, easy to implement	Burst at window boundary	APIs, Basic usage
Sliding Log	Store timestamps of requests in a log	Accurate	High memory usage	Strict rate limits
Sliding Window Counter	Weighted count of current + previous window	More accurate, lower memory	Slightly complex	General purpose
Token Bucket	Tokens added at constant rate. Each request consumes a token	Smooth traffic, allows bursts	Complex to tune	APIs, Network traffic
Leaky Bucket	Requests queued and processed at fixed rate	Smooth outgoing traffic	Requests may be delayed	Traffic shaping

Rate Limiting Best Practices

- ✓ Use appropriate algorithm for your use case
- ✓ Apply rate limiting as early as possible (at edge / gateway)
- ✓ Use meaningful limits (per user, IP, API key, plan)
- ✓ Return standard status code: 429 Too Many Requests
- ✓ Include Retry-After header
- ✓ Monitor and alert on rate limit breaches
- ✓ Allow higher limits for trusted clients / internal services

Example Response (429)

```

HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 60
{
  "error": "rate_limit_exceeded",
  "message": "Too many requests. Please try again later.",
  "retry_after_seconds": 60
}
  
```

Where to Implement?

1. API Gateway / Reverse Proxy (Nginx, Envoy, Kong)
2. Application Layer
3. Distributed Rate Limiter (Redis, Memcached)
4. CDN / Edge (Cloudflare, AWS WAF)

Identify by

- ✓ IP address
- ✓ User ID
- ✓ API Key
- ✓ Device ID
- ✓ Combination

Security Fundamentals (Must Have)

	Authentication Verify identity of users (Who are you?)		Authorization Verify permissions (What can you do?)
	Confidentiality Protect data from unauthorized access		Integrity Ensure data is accurate and not tampered
	Availability Ensure system is available when needed		Non-repudiation Actions should be traceable and verifiable

Common Security Threats

	Injection Attacks (SQL, NoSQL, OS)		Sensitive Data Exposure
	Cross Site Scripting (XSS)		Security Misconfiguration
	Cross Site Request Forgery (CSRF)		Broken Access Control (IDOR)
	Broken Authentication		Denial of Service (DoS / DDoS)

Security Best Practices

- ✓ Use HTTPS everywhere
- ✓ Validate and sanitize all inputs
- ✓ Use parameterized queries / ORM
- ✓ Implement proper authentication and authorization
- ✓ Enforce least privilege access
- ✓ Store passwords using strong hashing (bcrypt, Argon2)
- ✓ Use secure cookies (HttpOnly, Secure, SameSite)
- ✓ Keep dependencies and systems updated
- ✓ Log and monitor security events
- ✓ Backup data regularly and test restores

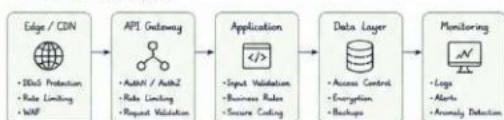
Rate Limiting Use Cases

Public APIs	Login Endpoints	E-commerce	Email / OTP	User Sign Up	File Downloads
Limit calls per IP / API key	Prevent brute force attacks	Prevent abuse during flash sales	Limit OTP / email requests	Prevent account creation abuse	Limit bandwidth usage

Useful HTTP Status Codes for Security

Code	Meaning	Use Case
401 Unauthorized	Authentication required	No valid credentials / token
403 Forbidden	Access denied	Authenticated but not allowed
429 Too Many Requests	Rate limit exceeded	Too many requests
400 Bad Request	Invalid request	Malformed / invalid input
500 Internal Server Error	Server error	Something went wrong

Defense in Depth (Layers)



Tools You Can Use

Category	Examples
Rate Limiting	NGINX (limit_req), Envoy, Kong, AWS API Gateway, Redis-based, Cloudflare Rate Limiting
Web Application Firewall	AWS WAF, Cloudflare WAF, ModSecurity
DDoS Protection	Cloudflare, AWS Shield, Google Cloud A
Security Scanning	OWASP ZAP, Snyk, Trivy, SonarQube
Secrets Management	Hackforce Vault, AWS Secrets Manager, Azure Key Vault

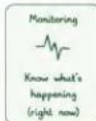
17. Monitoring + Logging + Fault Tolerance

• Why is this important?

- ✓ Detect issues early
- ✓ Understand system behavior
- ✓ Troubleshoot faster
- ✓ Ensure high availability & reliability
- ✓ Build resilient, user-friendly systems



• The 3 Pillars



• Goals

- ✓ High Availability
- ✓ Reliability
- ✓ Quick Detection
- ✓ Fast Recovery
- ✓ Minimal User Impact



1. Monitoring

Monitoring is collecting and analyzing metrics in real-time to understand the health and performance of the system.

Types of Metrics

Type	What it Measures	Examples
Infrastructure	CPU, Memory, Disk, Network	CPU %, Memory %, Disk IO, Network In/Out
Application	Requests, Errors, Latency	RPS, Error Rate, p95 Latency
Business	Business KPIs	Signups, Orders, Revenue
Custom	Domain specific metrics	Cache Hit Rate, Queue Length

Key Monitoring Components



Best Practices

- ✓ Monitor RED: Rate, Errors, Duration
- ✓ Set SLOs and Alerts
- ✓ Use dashboards for golden signals
- ✓ Avoid alert fatigue

Golden Signals (for services)

1. Latency
2. Traffic (RPS)
3. Errors
4. Saturation

2. Logging

Logging is recording events that help understand what happened.

Good Log Characteristics

- ✓ Structured (JSON)
- ✓ Consistent
- ✓ Useful (not too much, not too less)
- ✓ Include Context (request id, user id, time, service)
- ✓ Centralized and searchable

Example Log (JSON)

```
{
  "timestamp": "2024-05-24T10:20:30Z",
  "level": "ERROR",
  "service": "order-service",
  "message": "Payment failed",
  "request_id": "req_abc123",
  "user_id": "user_789",
  "order_id": "order_456",
  "error": "Insufficient balance"
}
```

Logging Flow



Best Practices

- ✓ Use log levels: DEBUG, INFO, WARN, ERROR
- ✓ Include context (request id, user id, trace id)
- ✓ Centralize logs
- ✓ Avoid logging sensitive data
- ✓ Set log retention & rotation
- ✓ Create alerts on important logs

3. Fault Tolerance

Fault tolerance ensures the system continues to operate even when components fail.

Common Failure Types



Key Principles

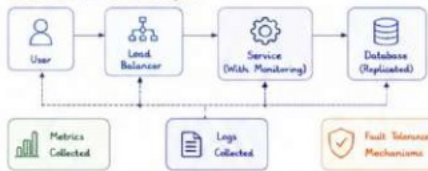
- ✓ Detect failures quickly
- ✓ Isolate failing components
- ✓ Recover automatically
- ✓ Degradate gracefully
- ✓ Design for retry and idempotency

Fault Tolerance Patterns

Pattern	Description	Use Case
Retry	Retry failed operations	Transient errors
Timeout	Fail fast if taking too long	Slow dependencies
Circuit Breaker	Stop calling failing service	Prevent cascading failures
Bulkhead	Isolate resources per component	Limit failure impact
Fallback	Provide alternative responses	Graceful degradation
Queue / Buffer	Absorb spikes and retry later	Traffic spikes
Idempotency	Make operations safe to retry	APIs, Payments

Putting It All Together

End-to-end flow in a resilient system



Example: Handling a Failure

- 1 Service becomes slow → Monitoring detects high latency → Alert triggered
- 2 Circuit Breaker opens → Further calls blocked → Fallback response returned
- 3 Logs captured for errors → Stored in central system → Team investigates
- 4 Issue fixed → Circuit closes automatically → Normal traffic resumes

Monitoring Tools

- 📊 Prometheus → Metrics collection & storage
- 📊 Grafana → Visualization & dashboards
- 🔔 Alertmanager → Alerting & routing
- 📊 Dataslog / New Relic → Full stack monitoring
- ☁️ CloudWatch → AWS monitoring service

Logging Tools

- 📁 ELK Stack (Elasticsearch, Logstash, Kibana)
- 📁 Loki + Grafana → Logs with low cost
- 📁 Fluentd / Filebeat → Log collection agents
- 📁 Splunk → Log analysis & monitoring
- ☁️ CloudWatch Logs → AWS logging service

Fault Tolerance Tools / Libraries

- 🔧 Resilience4j → Circuit breaker, retry, rate limiter
- 🔧 Hystrix (legacy) → Circuit breaker
- 🔧 Emoy → Service mesh, retries, timeouts
- 🔧 Kubernetes → Auto restart, self-healing
- ☁️ AWS / GCP Services → Auto scaling, HA

18. Real-World System Design Problem

What is System Design?

Designing scalable, reliable, and efficient systems to solve real-world problems at scale.

- ✓ Understand requirements
- ✓ Identify bottlenecks
- ✓ Design scalable architecture
- ✓ Handle millions of users
- ✓ Ensure reliability & fault tolerance

Design Process (High Level)



Things to Consider

- ✓ Scale (Users, Data, Traffic)
- ✓ Performance (Latency, Throughput)
- ✓ Availability & Reliability
- ✓ Consistency
- ✓ Cost
- ✓ Security
- ✓ Maintainability

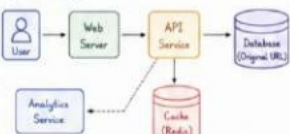


1. URL Shortener (like bit.ly)

Requirements

- Shorten long URLs
- Redirect to original URL
- Analytics (clicks, location, time)
- High availability, low latency

High Level Design



Key Components

- ✓ Hash function / Base62 encoding
- ✓ SQL/NoSQL DB to store mappings
- ✓ Cache for fast lookups
- ✓ CDN for static assets & global access

Scale Strategies

- ✓ Replicate DB (Read Replicas)
- ✓ Cache hot URLs
- ✓ Sharding / Partitioning
- ✓ Use CDN

2. Ride Sharing (like Uber)

Requirements

- Match riders with nearby drivers
- Real-time location tracking
- ETA & route optimization
- Payments, notifications
- High availability

High Level Design



Key Components

- ✓ User indexing (e.g., GeoHash)
- ✓ Real-time communication (WebSockets)
- ✓ Routing & ETA (maps, traffic)
- ✓ Microservices architecture

Scale Strategies

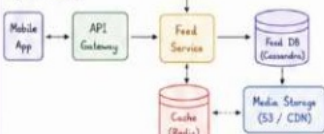
- ✓ Horizontal scaling of services
- ✓ Sharding by region
- ✓ Event-driven architecture
- ✓ Caching & CDN for static data

3. Social Media Feed (like Instagram)

Requirements

- Users can post, like, comment
- Users see personalized feed
- Support millions of users
- Real-time updates

High Level Design



Key Components

- ✓ Fan-out on write / Fan-in on read
- ✓ NoSQL DB for feed storage
- ✓ Media stored in object storage
- ✓ Caching for timelines

Scale Strategies

- ✓ Sharding (by user id)
- ✓ Pagination & lazy loading
- ✓ CDN for media
- ✓ Asynchronous processing

4. Video Streaming Platform (like Youtube)

Requirements

- Upload & store videos
- Stream with low buffering
- Support different resolutions
- Global availability

High Level Design



Key Components

- ✓ Upload to object storage
- ✓ Transcode to multiple resolutions
- ✓ CDN for fast delivery
- ✓ Metadata in DB

Scale Strategies

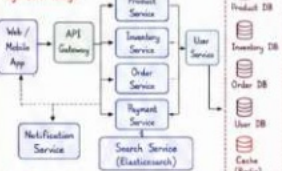
- ✓ Chunked upload
- ✓ Parallel transcoding
- ✓ Multi-CDN
- ✓ DB sharding

5. E-commerce System

Requirements

- Browse products, search
- Add to cart, place order
- Payments, order tracking
- High availability & consistency

High Level Design



Key Components

- ✓ Microservices
- ✓ Inventory management
- ✓ Reliable payments
- ✓ Search & recommendations

Scale Strategies

- ✓ DB replication & sharding
- ✓ Cache product/catalog data
- ✓ Async processing (queues)
- ✓ Auto-scaling services

6. Chat Application (like WhatsApp)

Requirements

- Real-time messaging
- Online status, typing...
- Deliver messages reliably
- Support millions of users

High Level Design



Key Components

- ✓ Websocket for real-time
- ✓ Message queue for reliability
- ✓ NoSQL DB for messages
- ✓ Presence & typing indicators

Scale Strategies

- ✓ Partition chats (by room id)
- ✓ Horizontal scaling
- ✓ Multi-region deployment
- ✓ Store & forward for offline

Common Data Stores

Store	Type	Best For	Examples
SQL DB	Relational	Transactions, consistency	MySQL, PostgreSQL
NoSQL DB	Document / Column / Key-Value	Large scale, flexible schema	MongoDB, Cassandra, DynamoDB
Cache	In-Memory	Fast access, session, markers	Redis, Memcached
Object Storage	Blob Storage	Files, images, videos	S3, GCS, Azure Blob
Search Engine	Search	Full-text search	Elasticsearch, OpenSearch
Queue / Stream	Messaging	Async processing	Kafka, RabbitMQ, SQS

Common Patterns

✓ Load Balancing	Distribute traffic across servers
✓ Caching	Reduce DB load, improve latency
✓ Replication	Data redundancy for high availability
✓ Sharding	Split data across multiple nodes
✓ CDN	Deliver content from edge locations
✓ Circuit Breaker	Prevent cascading failures
✓ Retry with Backoff	Handle transient failures
✓ Event Driven	Decouple services, async processing

Non-Functional Requirements

NFR	Description
Scalability	Handle growth in users, data, traffic
Availability	System should be up and running
Reliability	Failures rare, minimal downtime
Consistency	Data correctness across the system
Latency	Fast response time
Throughput	High number of requests per sec
Security	Protect data and ensure privacy
Maintainability	Easy to update and evolve

19. System Design Interview Questions +

Real Interview Questions. High Level Answers. Key Takeaways.

How to Approach Any System Design Question?

- 1. Clarify Requirements**
Ask questions. Understand the problem deeply.
- 2. Define Goals & Constraints**
Functional + Non-functional requirements.
- 3. High Level Design (HLD)**
Core components and flow.
- 4. Low Level Design (LLD)**
Deep dive into important components.
- 5. Scale, Optimize & Trade-offs**
Make it robust, scalable and cost-effective.

Common Non-Functional Requirements

- ✓ Scalability (to millions / billions)
- ✓ High Availability (99.9% or more)
- ✓ Low Latency
- ✓ Fault Tolerance
- ✓ Consistency
- ✓ Security
- ✓ Cost Effective
- ✓ Maintainability

Key Design Components (Most Systems Need)



1. Design a URL Shortener (like bit.ly)

Q: Design a service that shortens long URLs.

Answer (High Level)



Key Points

- Use Base62 encoding
- Store long URL → short code
- Cache lookups for speed
- Analytics: clicks, geo, referrers
- Scale with sharding + CDN

Tech: API, DB, Redis, CDN

2. Design a Ride Sharing System (like Uber)

Q: Design a system to match riders with drivers.

Answer (High Level)



Key Points

- Real-time location tracking
- Driver-rider matching
- ETA & route optimization
- Payments, notifications
- Highly available & scalable

Tech: APIs, Geo, WebSocket, Cache, DB

3. Design a Social Media Feed (like Instagram)

Q: Design a news feed system.

Answer (High Level)



Key Points

- Fan-out on write / read
- Store feed in DB / Cache
- Pagination for feed
- Media via CDN
- Real-time updates

Tech: DB, Cache, CDN

4. Design a Video Streaming Platform (like YouTube)

Q: Design a system to upload and stream videos.

Answer (High Level)



Key Points

- Upload to object storage
- Transcode to multiple formats
- Use CDN for delivery
- Metadata in DB
- Handle high traffic & storage

Tech: S3, CDN, Transcoding, DB

5. Design an E-commerce System

Q: Design an online e-commerce platform.

Answer (High Level)



Key Points

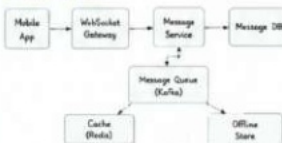
- Microservices architecture
- Inventory & stock management
- Payments & order tracking
- Search for products
- Scale independently

Tech: Microservices, DB, Cache, Search

Q. Design a Chat Application (like WhatsApp)

Q: Design a real-time chat system.

Answer (High Level)



Key Points

- Real-time messaging
- Store & sync messages
- Handle offline messages
- Presence, typing indicators
- Scale horizontally

Tech: WebSocket, MQ, DB, Cache

More Common System Design Questions

1. Design Netflix / OTT Platform
2. Design a Payment System (like PayPal)
3. Design a Notification System
4. Design a Search Engine (like Google Search)
5. Design a Rate Limiter
6. Design a Distributed File Storage System (like Dropbox)
7. Design a Leaderboard System
8. Design a News Aggregator (like Flipboard)
9. Design a IoT Data Ingestion System

One-Line High Level Answers

- Video upload, transcode, store, CDN delivery, recommendations.
- Payment flow, fraud check, transaction DB, settlement.
- Message queue, templates, delivery channels, retry logic.
- Crawling, indexing, ranking, search API, caching.
- Token bucket / leaky bucket, Redis, API Gateway.
- Chunking, replication, metadata store, versioning.
- Store scores, rank using sorted set, caching.
- Fetch from multiple sources, rank, cache, personalize.
- Device data ingestion, stream processing, storage, analytics.

Key Takeaways

- There is no single "right" design.
- Communicate & clarify requirements.
- Start simple, then add scale.
- Use the right components for the job.
- Discuss trade-offs & assumptions.
- Always think about failure & retry.
- Monitor, log and secure everything.
- Design for the future, not just today.



Things Interviewers Look For

- Problem solving approach



Quick Checklist Before You Design

- Have I clarified the requirements?
- Have I defined goals & constraints?

Pro Tips

- Use diagrams.